



Floating-Point Units on Espressif SoCs: Why (and when) they matter



COMMUNITY AUTHOR

Alberto Spagnolo

Senior firmware developer @ WeDo



 **ESPRESSIF AUTHOR**

Francesco Bez

Developer Relations at Espressif



In this article, you'll learn what an FPU is, why it's useful, which Espressif SoCs feature one, and how it impacts performance through a benchmark.

Introduction

When reading an Espressif SoC datasheet, you may have come across the acronym *FPU*. It often appears as just another item in the feature list, and while most of us have a vague idea that it relates to math, it is not always clear why it is actually useful.

As the name suggests, an FPU, or *Floating Point Unit*, is a part of the processor dedicated to working with floating-point numbers. These are the numbers with decimals that are essential for precise calculations in areas such as graphics, signal processing, or scientific workloads. On some CPUs, like the ESP32-S3, the FPU performs these operations directly in hardware, making them fast and efficient. On others, like the ESP32-C3, floating-point math is executed in



Adding to the confusion, discussions on forums and other resources often give conflicting answers about which Espressif cores include an FPU and which do not.

To understand why the presence or absence of an FPU matters, we first need to look at what floating-point numbers are, how they are represented in memory, and what makes their handling different from integers.

Understanding floats and their computation

When you store an **integer** in memory, each bit directly represents the number's value. For a 32-bit signed integer, the bits represent the magnitude using **two's complement**. For example, **5** is stored as:

```
00000000 00000000 00000000 0000101
```

and **-5** as:

```
11111111 11111111 11111111 11111011
```

Integers can exactly represent whole numbers, but they are limited in range. A 32-bit signed integer can only store values roughly between $-2,147,483,648$ and $2,147,483,647$. For numbers outside this range, or for fractional values, integers are not sufficient.

Floating-point numbers solve the limitations of integers by representing values in a format inspired by scientific notation: a combination of a **sign**, a **scale** (exponent) indicating magnitude, and **precision digits** (mantissa). This design allows numbers to span a wide range, including very large and very small values, as well as fractions.

Single precision, or **float**, uses 32 bits to balance range, precision, and memory usage, while double precision, or **double**, uses 64 bits to provide higher precision and an even wider range at the cost of additional memory.

Floats are essential whenever calculations require decimal values or a wide dynamic range, enabling applications from physics simulations to financial



The IEEE 754 standard

To ensure consistent floating-point behavior across platforms, the IEEE 754 standard defines exactly how floats are represented in memory. As mentioned, a single-precision float (`float`) uses 32 bits divided into three parts: a **sign bit**, an **exponent**, and a **mantissa** (also called the fraction).

The **sign bit** determines whether the number is positive or negative. A value of `0` indicates a positive number, while `1` indicates a negative number.

The **exponent** represents a power of two, determining the scale of the number. Positive exponents shift the value to larger numbers, while negative exponents allow representation of very small fractions. To encode both positive and negative exponents with unsigned bits, the standard uses a **bias**: the stored exponent equals the actual exponent plus a fixed bias (127 for single precision).

The **mantissa** encodes the significant digits of the number. In normalized numbers, there is an **implicit leading 1** before the binary point, which is not stored. The remaining bits store the fractional part in binary. For instance, a mantissa of `10000000000000000000000000000000` represents `1.5` because the leading 1 is implicit and the fractional part contributes `0.5` (1×2^{-1}). This scheme maximizes precision within the limited number of bits available.

To illustrate a complete float representation, consider the number `5.0` as a 32-bit float:

```
01000000 01000000 00000000 00000000
```

Breaking it down:

- **Sign bit** (`0`) → positive
- **Exponent** (`10000001`) → 129 in decimal, representing an actual exponent of 2 ($129 - 127 \text{ bias} = 2$)
- **Mantissa** (`01000000000000000000000000000000`) → encodes the significant digits 1.25

Putting all together:

$$(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exponent}-\text{bias})} = 5.0$$



do not yield correct results.

Performing float calculations

On a CPU with an FPU, operations like addition, multiplication, or square root execute directly in hardware using specialized instructions, making them fast and simple.

On a CPU **without an FPU**, like the ESP32-C3, there are no native float instructions. To perform a floating-point operation, you must emulate it using integer arithmetic, carefully handling the sign, exponent, and mantissa according to the IEEE 754 standard. For example, adding two floats **a** and **b** in software involves steps like:

1. Extract the sign, exponent, and mantissa from each number.
2. Align the exponents by shifting the mantissa of the smaller number.
3. Add or subtract the mantissas depending on the signs.
4. Normalize the result and adjust the exponent if necessary.
5. Pack the sign, exponent, and mantissa back into a 32-bit word.

A highly simplified illustration in C-like pseudocode:

```
uint32_t float_add(uint32_t a, uint32_t b) {
    int sign_a = a >> 31;
    int sign_b = b >> 31;
    int exp_a  = (a >> 23) & 0xFF;
    int exp_b  = (b >> 23) & 0xFF;
    uint32_t man_a = (a & 0x7FFFFFFF) | 0x800000; // implicit 1
    uint32_t man_b = (b & 0x7FFFFFFF) | 0x800000;

    // align exponents
    if (exp_a > exp_b) man_b >>= (exp_a - exp_b);
    else              man_a >>= (exp_b - exp_a);

    // add mantissas
    uint32_t man_r = (sign_a == sign_b) ? man_a + man_b : man_a - man_b;

    // normalize result (simplified)
    int exp_r = (exp_a > exp_b) ? exp_a : exp_b;
    while ((man_r & 0x800000) == 0) { man_r <<= 1; exp_r--; }
```



Even this simplified version is already much more work than a single FPU instruction, and handling all edge cases - overflow, underflow, NaNs (Not a Number), infinities, rounding - adds significant complexity.

This example illustrates why standard libraries provide dedicated routines for floating-point arithmetic. Libraries such as `libc` on Linux or `musl` include tested implementations of both low-level float operations and higher-level math functions, so developers don't have to implement them manually. Espressif relies on `newlibc`.

newlibc

On embedded RISC-V systems like the ESP32-C3, `newlibc` provides the standard C library support for floating-point operations. It is structured in two layers:

- Low level `libgcc`
- High level `libm`

At the low level, `libgcc` implements the primitive IEEE 754 operations in software. These routines use integer instructions and handle all the edge cases of floating-point arithmetic, including rounding, overflow, underflow, infinities, and NaNs. Typical routines include:

- **Arithmetic:** `__addsf3`, `__subsf3`, `__mulsf3`, `__divsf3` (single-precision), and `__adddf3`, `__muldf3` (double-precision).
- **Negation and absolute value:** `__negsf2`, `__abssf2`.
- **Comparisons:** `__eqsf2`, `__ltsf2`, `__gesf2`, etc.
- **Conversions:** between integers and floats or doubles, e.g., `__floatsisf`, `__fixsfsi`, `__fixunsdftsi`.

These helpers form the foundation for all floating-point operations on a CPU without an FPU, allowing higher-level routines to rely on them for correctness.

At a higher level, `libm` provides the familiar `<math.h>` functions, such as `sinf`, `cosf`, `sqrtf`, `expf`, and `logf`. These functions rely on the low-level helpers to perform their calculations correctly. For example, computing `sqrtf(2.0f)` on the ESP32-C3 is a software routine that uses iterative methods (like Newton–Raphson) together with integer arithmetic on the mantissa and exponent.



on doubles.

With an FPU, these operations can be executed directly as hardware instructions, making them fast and efficient. Without an FPU, newlibc ensures that all floating-point operations and math functions behave correctly according to IEEE 754, freeing developers from having to implement complex routines themselves.

FPU Performance and Precision in Espressif Cores

The main advantage of an FPU is speed. As we have seen earlier, floating-point arithmetic in software requires multiple integer operations to manipulate the sign, exponent, and mantissa of a number, whereas an FPU can produce the same result in a single instruction. This difference becomes especially significant in applications that involve **lots of numerical computation**, such as digital signal processing, machine learning, or 3D graphics.

FPU Precision Support

Not all FPUs are created equal. Some processors include an FPU that handles only **single-precision** (`float`) operations, while others provide a more capable FPU that supports **double-precision** (`double`) operations.

Cores with a **single-precision FPU** can execute `float` arithmetic directly in hardware, offering a significant speed advantage for typical embedded applications and digital signal processing tasks. However, `double` operations on these cores must still be emulated in software, which is slower.

Cores with a **full-precision FPU** handle both `float` and `double` natively, delivering fast, accurate results for applications that require high numerical precision, such as scientific calculations or complex control algorithms.

For Espressif cores, all currently available FPUs support **single-precision** only. This means that while `float` operations benefit from hardware acceleration, any `double` calculations are still handled in software, which may impact performance in computation-heavy applications.



cores include one. For a quick overview, Espressif provides a [SoC Product Portfolio](#) that lists the key features of each chip, including whether an FPU is present.

In practice, FPUs are found on cores designed for higher computational workloads, where fast floating-point arithmetic makes a noticeable difference. These cores include the **ESP32, ESP32-S3, ESP32-H4, and ESP32-P4**.

Core	FPU
ESP32	✓
ESP32-S2	
ESP32-S3	✓
ESP32-C2	
ESP32-C3	
ESP32-C5	
ESP32-C6	
ESP32-C61	
ESP32-H2	
ESP32-H4	✓
ESP32-P4	✓

To demonstrate the impact of an FPU, the next section presents a benchmark comparing the execution of the same floating-point workload on two Espressif cores, one with a hardware FPU and one without. This highlights the significant speed and efficiency advantages of hardware-accelerated floating-point operations.

Benchmark

To evaluate the performance improvement, we measure the average number of CPU cycles required by a core with an FPU and one without to execute the following functions:



- `cosf` and `cos` : compute the cosine of the number
- `float_mixed` and `double_mixed` : perform a more complex calculation involving division and trigonometry: `cos(sqrtf(value / 2.3 * 0.5 / value))`

Each function is executed 10,000 times in a loop, and the total cycle count is divided by 10,000 to obtain the average number of cycles per operation.

▼ Full benchmark code

```
#include <stdio.h>
#include <inttypes.h>
#include <stdio.h>
#include <math.h>

#include <esp_cpu.h>
#include <esp_system.h>
#include <esp_chip_info.h>
#include <esp_random.h>
#include <esp_log.h>

#define TAG          "test"
#define ITERATIONS  10000

static void float_test(const char* label, float seed, float (*function) (float))
{
    uint32_t start_cycles = esp_cpu_get_cycle_count();
    for (uint32_t i = 0; i < ITERATIONS; i++)
    {
        seed = function(seed);
    }
    uint32_t end_cycles = esp_cpu_get_cycle_count();

    printf(TAG ": %s %-10.10s average %"PRIu32 " cycles\n", label, "float:", (end_cycles - start_cycles) / ITERATIONS);
}

static void double_test(const char* label, double seed, double (*function) (double))
{
    uint32_t start_cycles = esp_cpu_get_cycle_count();
    for (uint32_t i = 0; i < ITERATIONS; i++)
    {
        seed = function(seed);
    }
    uint32_t end_cycles = esp_cpu_get_cycle_count();
```





```
typedef struct {
    float (*float_function) (float);
    double (*double_function) (double);
} test_t;

static void test(const char* label, test_t* test) {
    printf(TAG ": %s\n", label);

    double seed = 123456.789;

    if (test->float_function != NULL) {
        float_test(label, (float) seed, test->float_function);
    }

    if (test->double_function != NULL) {
        double_test(label, (double) seed, test->double_function);
    }

    printf("\n");
}

/** trivial functions */
static float float_sum10(float value) {
    return (value + 10);
}

static double double_sum10(double value) {
    return (value + 10);
}

static float float_div10(float value) {
    return (value / 10);
}

static double double_div10(double value) {
    return (value / 10);
}

/*
 * mixed calculations
 */
static float float_mixed(float value) {
    return cosf(sqrtf(value / 2.3f * 0.5f / value));
}
```





```
}

void app_main(void)
{
    esp_chip_info_t chip_info;
    esp_chip_info(&chip_info);

    const char *model;
    switch (chip_info.model) {
        case CHIP_ESP32:
            model = "ESP32";
            break;
        case CHIP_ESP32S2:
            model = "ESP32-S2";
            break;
        case CHIP_ESP32S3:
            model = "ESP32-S3";
            break;
        case CHIP_ESP32C3:
            model = "ESP32-C3";
            break;
        case CHIP_ESP32H2:
            model = "ESP32-H2";
            break;
        case CHIP_ESP32C2:
            model = "ESP32-C2";
            break;
        case CHIP_ESP32C6:
            model = "ESP32-C6";
            break;
        default:
            model = "<UNKNOWN>";
            break;
    }

    printf("\n");
    printf(TAG ": CHIP: %s\n", model);
    printf("\n");

    test_t sum10 = {
        .float_function = float_sum10,
        .double_function = double_sum10,
    };
    test("SUM", &sum10);

    test_t div10 = {
```



```
test("DIV", &div10);

test_t cosine = {
    .float_function = cosf,
    .double_function = cos,
};
test("COS", &cosine);

test_t mixed = {
    .float_function = float_mixed,
    .double_function = double_mixed,
};
test("MIX", &mixed);
}
```

The results are collected in the following table.

		SUM		DIV		COS		MIX	
	FPU	float	double	float	double	float	double	float	double
ESP32C3	✗	100	122	102	133	2377	3560	3659	6000
ESP32S3	✓	25	70	69	75	121	1619	312	3000
Delta cycles		-75	-52	-33	-58	-2256	-1941	-3347	-3000
Saving		75%	43%	32%	44%	95%	55%	91%	50%

As expected, single precision float calculation show the biggest saving in machine cycles, with a peak of 95% reached with the cosine function. This means that if your application requires intense vector math, an FPU can come quite handy.

Conclusion

In this article, we explored floating-point numbers, how CPUs perform these calculations with and without an FPU, and which Espressif cores include one. Benchmarks showed that cores with an FPU executed operations, especially single-precision, much faster, demonstrating the clear performance advantage





0 reactions



0 comments

Write

Preview

Aa

Sign in to comment

M↓

 Sign in with GitHub

Related





Debugging with ESP-IDF VS Code extension: Part 2

12 June 2025 · Updated: 11 February 2026
· 8 mins

[Debugging](#)
[ESP-IDF](#)
[ESP32-C3](#)
[Esp32-S3](#)
[VSCode](#)

This two-part guide shows how to set up VS Code with the ESP-IDF extension to debug Espressif boards using JTAG. In this second part, we will debug a simple project using gdb through...



Debugging with ESP-IDF VS Code extension: Part 1

20 May 2025 · Updated: 10 February 2026
· 9 mins

[Debugging](#)
[ESP-IDF](#)
[ESP32-C3](#)
[Esp32-S3](#)

This two-part guide shows how to set up VS Code with the ESP-IDF extension to debug Espressif boards using JTAG. This first part covers the debugging process, hardware setup and...



Espressif part numbers explained: A complete guide - Modules

19 March 2025 · 7 mins

[ESP32](#)
[ESP32-S2](#)
[Esp32-S3](#)
[ESP32-H2](#)
[ESP32-C3](#)

